

VDTI Service Hub — API Integration Guide

Base URL (Production): <https://vaccumapi-production.up.railway.app>

Base URL (Local): <http://localhost:3000>

Swagger UI: {BASE_URL}/api-docs

1. Authentication

All API requests (except login, register, forgot/reset password) require a JWT token.

Login

POST /api/auth/login

Content-Type: application/json

```
{
  "email": "admin@vdti.com",
  "password": "yourpassword"
}
```

Response:

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": { "id": 1, "role": "admin", "first_name": "Arjun" }
}
```

Using the token

Add this header to **every** subsequent request:

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

2. Role-Based Access

Role	Access Level
admin	Full access to everything
manager	Most routes except user management
engineer	Jobs, clients, technicians (read + create)
technician	Read all + advance own jobs + submit reports
labour	Read only (jobs, clients, technicians)

3. Modules & Endpoints

Auth — /api/auth

Method	Endpoint	Auth	Description
POST	/login	✗	Login (email or phone + password)
POST	/register	✗	Register new user
GET	/me	✓	Get current user profile
POST	/forgot-password	✗	Request password reset link
POST	/reset-password	✗	Reset password with token
POST	/change-password	✓	Change own password

Users — /api/users (admin only)

Method	Endpoint	Description
GET	/	List all users
POST	/	Create user
GET	/:id	Get user by ID
PUT	/:id	Update user

DELETE	<code>/:id</code>	Deactivate user (soft delete)
--------	-------------------	-------------------------------

Technicians — `/api/technicians`

Method	Endpoint	Auth	Description
POST	<code>/login</code>	✗	Technician login
GET	<code>/</code>	✓	List technicians
POST	<code>/</code>	✓ admin/manager/engineer	Add technician
GET	<code>/:id</code>	✓	Get technician + recent jobs
PUT	<code>/:id</code>	✓ admin/manager/engineer	Update technician
DELETE	<code>/:id</code>	✓ admin/manager	Delete technician

Clients — `/api/clients`

Method	Endpoint	Description
GET	<code>/</code>	List clients
POST	<code>/</code>	Create client
GET	<code>/:id</code>	Get client + stats
PUT	<code>/:id</code>	Update client
DELETE	<code>/:id</code>	Delete client

Jobs — `/api/jobs`

Method	Endpoint	Description
GET	<code>/</code>	List jobs (filter by status, priority, client, etc.)
POST	<code>/</code>	Raise new job
GET	<code>/:id</code>	Get job + images + linked reports
PUT	<code>/:id</code>	Update job details

PATCH	<code>/:id/status</code>	Advance job status (pipeline)
DELETE	<code>/:id</code>	Delete job (admin only)
POST	<code>/:id/images</code>	Add image(s) to job
DELETE	<code>/:id/images/:imageId</code>	Remove image from job

Job status pipeline (forward only):

Raised → Assigned → In Progress → Closed

Reports — `/api/reports`

Method	Endpoint	Description
GET	<code>/</code>	List reports
POST	<code>/</code>	Submit report (status always Pending)
GET	<code>/:id</code>	Get report + images
PATCH	<code>/:id/status</code>	Approve or Reject (admin only)
POST	<code>/:id/images</code>	Add image(s) to report
DELETE	<code>/:id/images/:imageId</code>	Remove image from report

AMC Contracts — `/api/amc`

Method	Endpoint	Description
GET	<code>/</code>	List AMC contracts
POST	<code>/</code>	Create contract
GET	<code>/expiring</code>	Contracts whose reminder fires today (for cron)
GET	<code>/:id</code>	Get contract + services
PUT	<code>/:id</code>	Update contract
DELETE	<code>/:id</code>	Delete contract (admin)

Email Settings — `/api/email-settings` (admin only)

Method	Endpoint	Description
GET	<code>/</code>	Get SMTP config + notification toggles
PUT	<code>/</code>	Save/update SMTP config + toggles (upsert)
POST	<code>/test</code>	Send a test email

File Upload — `/api/upload`

Method	Endpoint	Description
POST	<code>/</code>	Upload image(s) → returns public URLs
DELETE	<code>/:id</code>	Delete uploaded file

Dashboard — `/api/dashboard`

Method	Endpoint	Description
GET	<code>/</code>	All KPIs, charts, and recent jobs in one call

4. File Upload Flow

Two-step process:

Step 1 — Upload the file:

POST `/api/upload?entity_type=job&entity_id=JOB-0001`

Authorization: Bearer <token>

Content-Type: multipart/form-data

images: [file1.jpg, file2.png]

Response:

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "file_url":
        "https://vaccumapi-production.up.railway.app/uploads/1714012345678_file1.jpg",
    }
  ]
}
```

```

    "original_name": "file1.jpg",
    "mime_type": "image/jpeg"
  }
]
}

```

Step 2 — Link to job:

POST /api/jobs/JOB-0001/images
 Authorization: Bearer <token>
 Content-Type: application/json

```

{
  "file_name": "file1.jpg",
  "file_url": "https://vaccumapi-production.up.railway.app/uploads/1714012345678_file1.jpg",
  "mime_type": "image/jpeg"
}

```

Pass an **array** `[ {...}, {...} ]` to add multiple at once.

5. WebSocket Real-Time Notifications

Connect:

```

// Browser / React Native
const ws = new WebSocket('wss://vaccumapi-production.up.railway.app/ws');

// Local dev
const ws = new WebSocket('ws://localhost:3000/ws');

```

Step 1 — Authenticate (within 10 seconds of connecting):

```

ws.onopen = () => {
  ws.send(JSON.stringify({
    type: 'auth',
    token: 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...'
  }));
};

```

Step 2 — Listen for events:

```

ws.onmessage = (event) => {

```

```

const msg = JSON.parse(event.data);
console.log(msg.event, msg.data);

switch (msg.event) {
  case 'connected':    // Authentication confirmed
  case 'job_raised':   // New job created
  case 'job_status':   // Job status changed (e.g. → Closed)
  case 'report_submitted': // New report submitted
  case 'report_reviewed': // Report approved/rejected
  case 'amc_expiring':  // AMC nearing expiry
  case 'notification':  // Generic message
  case 'pong':          // Response to ping
  case 'error':        // Error message
}
};

```

Keep-alive ping (send every 30 seconds):

```

setInterval(() => {
  if (ws.readyState === WebSocket.OPEN) {
    ws.send(JSON.stringify({ type: 'ping' }));
  }
}, 30000);

```

React Native example:

```

import { useEffect, useRef } from 'react';

export function useWebSocket(token) {
  const ws = useRef(null);

  useEffect(() => {
    ws.current = new WebSocket('wss://vaccumapi-production.up.railway.app/ws');

    ws.current.onopen = () => {
      ws.current.send(JSON.stringify({ type: 'auth', token }));
    };

    ws.current.onmessage = (e) => {
      const msg = JSON.parse(e.data);
      if (msg.event === 'job_raised') {
        // Show notification, refresh list, etc.
      }
    };

    return () => ws.current?.close();
  });
}

```

```
}, [token]);  
}
```

6. Email (SMTP) Configuration

Option A — .env file (simplest):

```
SMTP_HOST=smtp.gmail.com  
SMTP_PORT=587  
SMTP_USER=your_gmail@gmail.com  
SMTP_PASS=xxxx xxxx xxxx xxxx # Gmail App Password  
SMTP_FROM_NAME=VDTI Service Hub
```

Option B — via API (persists in DB, overrides .env):

```
PUT /api/email-settings  
Authorization: Bearer <admin_token>  
Content-Type: application/json
```

```
{  
  "smtp_host": "smtp.gmail.com",  
  "smtp_port": 587,  
  "from_email": "your_gmail@gmail.com",  
  "from_name": "VDTI Service Hub",  
  "smtp_password": "xxxx xxxx xxxx xxxx",  
  "notifications": {  
    "job_raised": true,  
    "job_assigned": true,  
    "job_completed": true,  
    "report_approved": false,  
    "amc_renewal": true,  
    "quotation_sent": false  
  }  
}
```

Getting a Gmail App Password: Google Account → Security → 2-Step Verification → App Passwords → Create → Copy the 16-character password.

7. Dashboard Response Shape

```
{  
  "success": true,
```

```

"data": {
  "stats": {
    "active_jobs": 4,
    "total_clients": 5,
    "active_technicians": 3,
    "total_technicians": 5,
    "revenue_approved": 1380000,
    "pending_reports": 1,
    "active_amc_count": 2,
    "mom_active_jobs": 12,
    "mom_revenue": 18,
    "mom_clients": 5
  },
  "job_status_breakdown": [
    { "status": "Raised", "count": 4 },
    { "status": "Assigned", "count": 6 },
    { "status": "In Progress", "count": 8 },
    { "status": "Closed", "count": 22 }
  ],
  "monthly_stats": [
    { "month": "Aug 2024", "jobs_raised": 20, "jobs_completed": 15, "revenue": 280000 }
  ],
  "revenue_trend": [
    { "month": "Aug 2024", "revenue": 280000 }
  ],
  "quick_overview": {
    "jobs_this_month": { "value": 24, "target": 30 },
    "jobs_completed": { "value": 19, "target": 24 },
    "active_technicians": { "value": 3, "target": 5 },
    "amc_active": { "value": 2, "target": 3 }
  },
  "recent_jobs": [
    {
      "id": "JOB-001",
      "title": "HVAC Annual Servicing",
      "client_name": "TechCorp Solutions",
      "status": "Closed",
      "priority": "High",
      "amount": 18500
    }
  ]
}

```

8. Pagination

All list endpoints support:

?page=1&limit=20

Response always includes:

```
{
  "pagination": {
    "total": 87,
    "page": 1,
    "limit": 20,
    "total_pages": 5
  }
}
```

9. Error Responses

All errors follow this shape:

```
{
  "success": false,
  "error_code": "JOB_NOT_FOUND",
  "message": "Job not found.",
  "details": null
}
```

HTTP Code	Meaning
400	Validation error / bad input
401	Missing or invalid JWT token
403	Authenticated but insufficient role
404	Resource not found
409	Conflict (duplicate / blocked delete)
500	Internal server error